

Changer le style de police dans une application Flutter

Utilisation du package `google_fonts` — Démarche et explication du code

1. Objectif

L'objectif était d'afficher un texte dans une application Flutter en utilisant une police personnalisée (Google Fonts), avec un style particulier : taille, graisse (gras), inclinaison (italique), couleur et espacement des lettres. Nous avons utilisé le package `google_fonts` plutôt qu'une police locale (fichier `.ttf`), car il évite les erreurs de configuration liées au fichier `pubspec.yaml` et permet de changer de police en une seule ligne.

2. Démarche suivie, étape par étape

Étape 1 — Ajouter la dépendance

Dans le fichier `pubspec.yaml`, sous la section `dependencies:`, nous avons ajouté la ligne `google_fonts: ^6.0.0`. Cela indique à Flutter de télécharger ce package externe qui donne accès à toutes les polices du catalogue Google Fonts.

Étape 2 — Récupérer le package

Nous avons exécuté la commande `flutter pub get` dans le terminal. Cette commande lit le fichier `pubspec.yaml`, télécharge les dépendances déclarées et les rend disponibles dans le projet.

Étape 3 — Importer le package dans le code

Dans `main.dart`, nous avons ajouté la ligne `import 'package:google_fonts/google_fonts.dart';` tout en haut du fichier, en plus de l'import habituel de Flutter Material.

Étape 4 — Appliquer la police au widget Text

Nous avons remplacé la propriété `fontFamily` classique d'un `TextStyle` par un appel direct à une méthode du package, par exemple `GoogleFonts.roboto(...)`, qui retourne elle-même un objet `TextStyle` déjà configuré avec la bonne police.

Étape 5 — Résoudre les erreurs de sauvegarde et relancer

Nous avons résolu un conflit d'enregistrement du fichier (« Resolve save conflict »), sauvegardé le fichier proprement, puis relancé l'application avec `flutter run -d chrome` pour voir le résultat dans le navigateur.

3. Code final utilisé (main.dart)

```
import 'package:google_fonts/google_fonts.dart';
import 'package:flutter/material.dart';

void main() {
  runApp(
    MaterialApp(
      home: Scaffold(
        body: Center(
          child: Text(
            'Hello Word',
            style: GoogleFonts.roboto(
              fontSize: 20,
              fontWeight: FontWeight.bold,
              fontStyle: FontStyle.italic,
              color: Colors.deepOrange,
              letterSpacing: 1.2,
            ),
          ),
        ),
      ),
    ),
  );
}
```

4. Explication ligne par ligne

Ligne de code	Explication
import 'package:google_fonts/google_fonts.dart';	Importe le package google_fonts, indispensable pour utiliser la classe GoogleFonts
import 'package:flutter/material.dart';	Importe la bibliothèque Material de Flutter, qui fournit les widgets de base (MaterialApp, Scaffold, Center, Text, etc.)
void main() { runApp(...); }	Point d'entrée de toute application Flutter. La fonction main() est exécutée en premier.
MaterialApp(...)	Widget racine qui configure l'application entière (thème, navigation, etc.). Il doit englober tout le contenu de l'application.
home: Scaffold(...)	Scaffold fournit la structure visuelle standard d'un écran Material (zone pour lAppBar, zone de contenu, etc.).
body: Center(child: ...)	Center est un widget qui centre son enfant horizontalement et verticalement au milieu de l'écran.
Text('Hello Word', style: ...)	Widget Text qui affiche la chaîne de caractères indiquée. Le paramètre style permet de personnaliser l'apparence du texte.
style: GoogleFonts.roboto(...)	Au lieu d'utiliser TextStyle(fontFamily: 'Roboto'), on appelle directement la méthode roboto de GoogleFonts.
fontSize: 20	Définit la taille du texte en pixels logiques (ici 20).
fontWeight: FontWeight.bold	Définit la graisse de la police : ici en gras (bold). D'autres valeurs existent, comme FontWeight.normal.
fontStyle: FontStyle.italic	Incline le texte en italique. L'autre valeur possible est FontStyle.normal.
color: Colors.deepOrange	Définit la couleur du texte, ici un orange soutenu, à partir de la palette de couleurs prêtes à l'emploi de Flutter.
letterSpacing: 1.2	Ajoute un espacement supplémentaire de 1.2 pixels logiques entre chaque lettre du texte.

5. Comment tester d'autres polices

Pour changer de police, il suffit de remplacer le nom de la méthode après `GoogleFonts.` par une autre police disponible dans le catalogue Google Fonts. Tous les autres paramètres (`fontSize`, `color`, etc.) restent identiques.

```
GoogleFonts.poppins(fontSize: 24, color: Colors.red);  
GoogleFonts.pacifico(fontSize: 24, color: Colors.red);  
GoogleFonts.lobster(fontSize: 24, color: Colors.red);  
GoogleFonts.montserrat(fontSize: 24, fontStyle: FontStyle.italic);
```

Après modification, il suffit de sauvegarder le fichier (Ctrl+S) : le hot reload de Flutter met à jour l'affichage automatiquement, sans redémarrer toute l'application.

6. Points de vigilance rencontrés pendant la manipulation

- | Toujours vérifier l'indentation exacte (espaces, pas de tabulations) dans `pubspec.yaml` : ce fichier est un fichier YAML, très strict sur la syntaxe.
- | Ne jamais dupliquer la clé `flutter` : dans `pubspec.yaml` : elle ne doit apparaître qu'une seule fois dans tout le fichier.
- | En cas de message « Resolve save conflict », toujours vérifier quelle version du fichier on souhaite conserver avant d'écraser l'autre.
- | Arrêter proprement l'exécution en cours (Ctrl+C dans le terminal) avant de relancer l'application, pour éviter les conflits entre deux instances.
- | La commande `flutter analyze` permet de vérifier rapidement l'ensemble du projet et de repérer les erreurs restantes.

7. Désactiver la rotation de l'écran

Pour forcer l'application à rester dans une seule orientation (par exemple toujours en portrait, même si l'utilisateur tourne son téléphone), on utilise la classe `SystemChrome` du module `flutter/services.dart`, à l'intérieur de la fonction `main()`.

```
import 'package:flutter/material.dart';
import 'package:flutter/services.dart';
import 'package:google_fonts/google_fonts.dart';

void main() {
  WidgetsFlutterBinding.ensureInitialized();
  SystemChrome.setPreferredOrientations([
    DeviceOrientation.portraitUp,
  ]).then((_) {
    runApp(
      MaterialApp(
        home: Scaffold(
          body: Center(
            child: Text(
              'Hello Word',
              style: GoogleFonts.roboto(
                fontSize: 20,
                fontWeight: FontWeight.bold,
                fontStyle: FontStyle.italic,
                color: Colors.deepOrange,
                letterSpacing: 1.2,
              ),
            ),
          ),
        ),
      ),
    );
  });
}
```



Explication des nouvelles lignes

Ligne de code	Explication
<code>import 'package:flutter/services.dart'</code>	Donne accès aux classes <code>SystemChrome</code> et <code>DeviceOrientation</code> , nécessaires pour contrôler l'orientation.
<code>WidgetsFlutterBinding.ensureInitialized();</code>	Initialise le moteur Flutter avant d'appeler une API native. C'est obligatoire dès qu'on utilise une API native.
<code>SystemChrome.setPreferredOrientations([</code>	Indique au système d'exploitation la ou les orientations autorisées pour l'application.
<code>DeviceOrientation.portraitUp</code>	Valeur qui impose l'orientation portrait, haut vers le haut. C'est la seule orientation autorisée.
<code>.then((_) { runApp(...); })</code>	Attend que la contrainte d'orientation soit appliquée avant de démarrer réellement l'application.

Autres orientations possibles

```
DeviceOrientation.portraitUp    // portrait normal
DeviceOrientation.portraitDown  // portrait à l'envers
DeviceOrientation.landscapeLeft // paysage, incliné à gauche
DeviceOrientation.landscapeRight // paysage, incliné à droite
```

Il est possible d'autoriser plusieurs orientations à la fois en les ajoutant toutes dans la liste, par exemple `[DeviceOrientation.portraitUp, DeviceOrientation.portraitDown]`.

Remarque : sur le web (Chrome), cette contrainte n'a pas toujours d'effet visible, car la fenêtre du navigateur n'est pas « tournée » comme un vrai téléphone. Elle s'applique surtout sur un appareil Android/iOS réel ou un émulateur.

Document généré comme support d'explication pour la personnalisation des polices et le contrôle de l'orientation d'écran dans une application Flutter.